TECHNICAL REPORT, SECOND EDITION

ClaimGuard
Server Deployment and Test Report

What changed since the first report, how it performs now, what failed, and what to do next

Run	`server_20261101h` (about 42 minutes, 1 November 2026); earlier run `server_20261031` used for comparison
Address	https://claimguard.kernelsi.com
Server	Ubuntu 26.04 in Proxmox, AMD Ryzen 7 5700X, 16 GiB RAM, NVIDIA RTX 3080 (10 GB)
Models	Explanations: gemma3:4b. Photo reading: richardyoung/unlimited-ocr. Both served by Ollama, one at a time on the GPU.
Data	Synthetic claims only. No real patient data was used.
Branch	khalil_test, github.com/PublisherX02/Claim_Guard

Contents

1. Executive summary
2. What was deployed and how, and why the model server changed from vLLM to Ollama
3. How the tests work
4. Results, experiment by experiment (S1 to S7, S0)
5. Findings and recommendations
6. Problems met and how each was solved
7. Checklist before real use
8. What this test does not cover
9. Appendix: files, commands, local settings and glossary

1. Executive summary

ClaimGuard is installed and working on the server with every feature turned on: AI explanations of claim decisions, reading of claim documents (PDF, scan and, now with a vision model, phone photos) and search of the rulebook by meaning. It is reachable at **https://claimguard.kernelsi.com** through Cloudflare. Since the first report the project was updated to its newest version and the **explanation model was moved from vLLM to Ollama**, so that the vision model for photos can share the one 10 GB graphics card (section 2.5 explains why). The test suite ran again in full. The script counted **20 of 27 measures as passed**. The audit check (S7) first met an HTTP 502 from the API, which is itself a finding; it was repeated a few minutes later and the audit chain and the ledger verify.

210 / min
AI explanations (target 200; 376 with vLLM)

4,140 / min
claims through the pipeline (target 200; 11,721 earlier)

0 lost
claims lost in 5 restarts

90.0 %
document fields read exactly (86.4 % before)

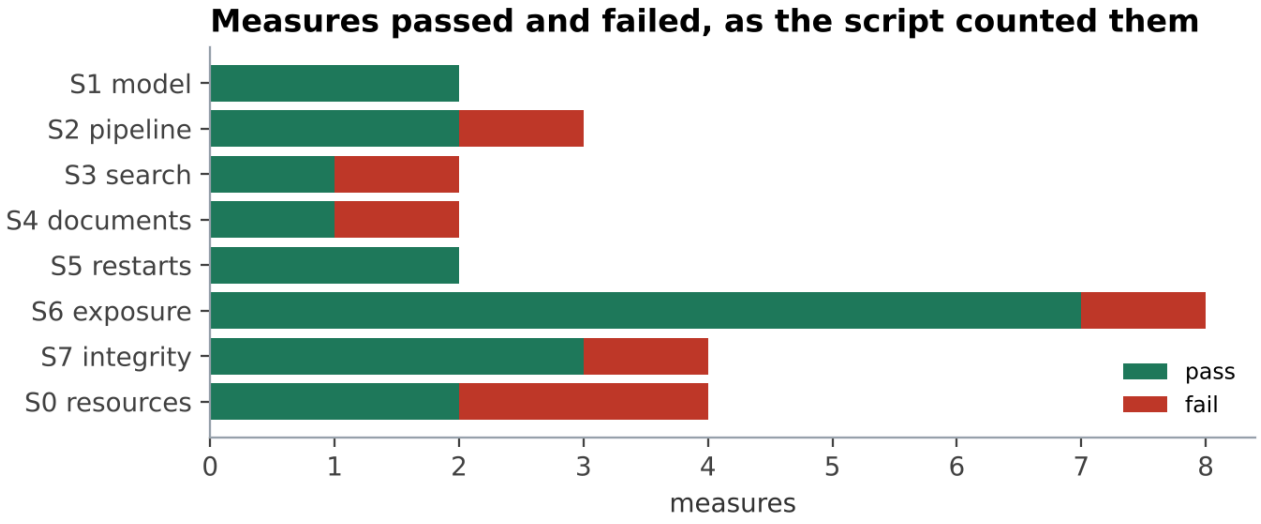


Figure 1. Measures passed and failed per experiment, as the test script counted them. S7 shows the result of the repeated check.

What improved

- **Documents.** Image-only PDFs and scans are now read at 100 % of fields (97.3 % and 98.2 % before). Phone photos rose from 50 % to 60 % of fields, and the share of photos with every field right from 20 % to 60 %. Overall: 90.0 % of fields (was 86.4 %).
- **One card, both models.** The explanation model and the vision model now share the graphics card without any manual switching. GPU memory in use peaked at 5.8 GiB of 10 GiB (58 %); the vLLM setup kept 7.7 GiB reserved at all times.
- **Steady load.** 3,000 claims at 100 a minute for 30 minutes: no failed batch. The failed batch of the first run did not repeat (its cause stays unknown).

What still works well

- **Safety of data.** Five services were restarted again while claims arrived. Every acknowledged claim was still there afterwards (0 lost).
- **Integrity.** The audit chain (19,197 events, signed anchor) and the ledger (77 checkpoints, 19,199 log rows) verify.
- **Security from the outside.** 7 of 8 checks pass; the eighth is the known Cloudflare port 8080 artefact. Only ports 80 and 443 listen on the server.

What needs work

Finding	Type	Importance
The API answered HTTP 502 for a moment at the end of the document test (first attempt of S7)	Real, probably its memory limit	**High**
Health probes: "flow" degraded, "timestamp" unknown	Flow: cause not checked. Timestamp: no external timestamp service	**Medium**
Phone photos are read correctly in 60 % of fields (target 95 %)	Real, improved from 50 %	**High for document intake**
The API sits at its 4 GiB memory limit and the dashboard and health panel take about 2 s even when idle	Real, grows with stored claims (about 15,000)	**High**
Explanations: 210 a minute, only 5 % above the target (376 with vLLM)	Real, a cost of the change to Ollama	**Medium**
The Ollama container used 99 % of its 5 GiB memory limit	Real, may be file cache (not checked)	**Medium**
One of five off-topic questions in search was answered instead of refused	Real, tunable (unchanged)	**Medium**
Search speed test blocked by the rate limit (HTTP 429)	Test artefact	**Low, re-run with the limit raised**
Port 8080 appeared open	Test artefact (Cloudflare answers on that port)	**None**

2. What was deployed and how

2.1 The machine

Item	Value
Host	khalil, a Proxmox container running Ubuntu 26.04 LTS (kernel 7.0.14)
CPU	AMD Ryzen 7 5700X, 8 threads available
Memory	16 GiB (raised from 12 GiB after a build froze the server)
GPU	NVIDIA GeForce RTX 3080, 10,240 MiB, driver 615.71.09
Disk	99 GB. It filled up twice on 1 November (build cache, then core dumps); see section 6. 25 GB free at the end of the run
Software	Docker 29.9.2, Docker Compose 5.6.1

2.2 The services

ClaimGuard runs as eight containers managed by Docker Compose. Each does one job:

Container	Role	Notes
proxy (Caddy)	Receives all web traffic, handles HTTPS	The only service that listens on the network (ports 80 and 443)
api	The ClaimGuard API and the web console	Written in Rust, memory limit raised to 4 GiB
worker	Processes the queue of claims	Rust, memory limit 3 GiB
mongo	Database	MongoDB 7
redis	Queue and short-term storage	Redis 7
ollama	Runs both AI models on the GPU, one at a time	gemma3:4b (explanations) and richardyoung/unlimited-ocr (photos); replaces the vllm container
embed	Search by meaning	bge-m3 model, runs on the CPU
reader	Reads documents	Tesseract plus the vision model; the two readings are cross-checked

2.3 How a visitor reaches it

A browser opens `claimguard.kernelsi.com`. Cloudflare answers first, then forwards the request to the server, where Caddy passes it to the API. A Cloudflare rule sends anyone who types `http://` to `https://`. Inside the server, no other service is exposed.

2.4 Settings that must survive every update

Four files hold the changes that make this server work. A project update must not overwrite them. The update to the new project version followed this rule: a backup was taken first, and the new files were copied in with these excluded.

File	Why it matters
`.env`	Passwords, keys, model settings, the site name
`deploy/Caddyfile.internal`	Sends the real domain name to the API (`header_up Host`) and sets a default certificate name; without it the login fails with "That request came from another site"
`deploy/rust/compose.gpu.yml`	Gives the GPU containers access to the card inside the container, mounts extra GPU libraries, sets memory limits
`deploy/rust/compose.local.yml`	**New.** This server's own settings: the Ollama service, the address of the explanation model (Ollama instead of vLLM), the vision model for the reader, the 4 GiB API limit. The project never overwrites it and keeps it out of Git (document 47, "Keeping local changes across updates"). Its content is in section 9.4

2.5 Why the explanation model moved from vLLM to Ollama

The short answer. The vision model that reads phone photos needs several gigabytes of GPU memory while it works, and vLLM keeps most of the card for itself. Both could not run on the 10 GB card at the same time. Serving both models through Ollama, which loads one at a time and swaps them automatically, solved this. It is also how the project was tested on the developer's 8 GB laptop.

What was observed.

- With Tesseract alone, phone photos were read correctly in only 50 % of fields (first report). The project documentation shows a vision model lifting photos to 88.1 % on the build PC, so the vision model was added.
- vLLM was configured to reserve 85 % of the card (7.7 GiB measured). The vision model needs about 2.8 GB for its weights and a working buffer of about 2.4 GB for a large photo. Only about 1.4 GiB was left. The vision model ran out of GPU memory and crashed: all 80 photos in a direct test returned HTTP 500, and the reader silently fell back to Tesseract alone, so the photo score did not change (50 %).
- Each crash also wrote a core dump inside the Ollama container. About 80 crashes filled 30 GB and, together with the earlier build cache, filled the disk twice.
- With vLLM stopped, the same vision model read photos in about 2 seconds each and the photo score rose to 60 %. So the model works; the card was the limit.

The options that were weighed.

Option	Result
Keep vLLM and add the vision model beside it	**Out of GPU memory (HTTP 500 on every photo). Not usable on a 10 GB card; document 47 also says the project's vision profile does not fit beside Gemma on 10 GB**
Give vLLM less of the card	**Not enough. A first try at 55 % had already failed with "no memory for the KV cache"; and the vision model alone used about 5.6 GiB of the card when loaded, so the two together exceed 10 GiB. (A value of 70 % was considered and not tried, because this arithmetic does not close.)**
Stop vLLM by hand whenever photos are imported	**Works (photos at 60 %, 2 s each) but needs a person to switch the models by hand every time**
Run the vision model on the CPU	**2 to 4 minutes per photo; 39 of 40 documents failed in that test because the test client gave up after 60 seconds**
Serve both models through Ollama (chosen)	**One model on the card at a time, swapped automatically. Gemma passed the rulebook check (15 of 15) and the photo test ran after it, then Gemma passed again**

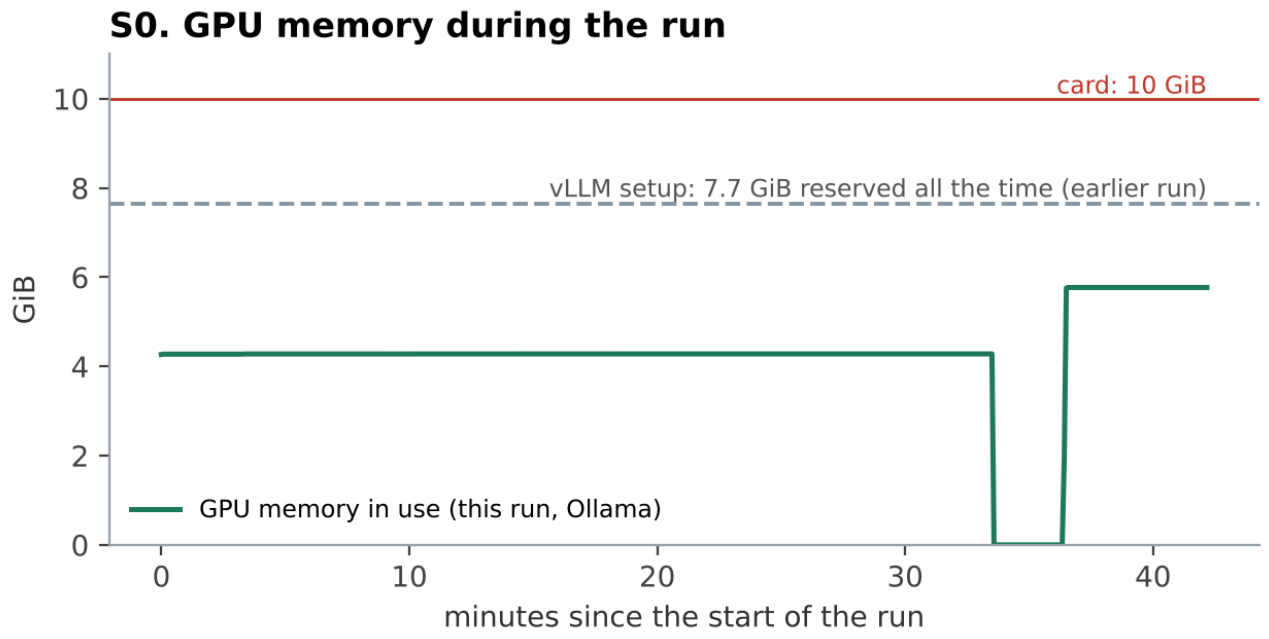


Figure 2. GPU memory in use during this run (green) against the vLLM setup (dashed grey, 7.7 GiB reserved at all times) and the 10 GiB card. The brief dip to zero is a moment with no model loaded; the step to 5.8 GiB is the vision model being loaded for the photo test.

What it costs.

	vLLM setup (first run)	Ollama setup (this run)
Explanation model	google/gemma-3-4b-it, fp8 weights	gemma3:4b, Ollama's 4-bit build (3.4 GB)
GPU memory	7.7 GiB reserved at all times	4.3 GiB with Gemma; peak 5.8 GiB with the vision model
Best explanations per minute	**376**	**210 (target 200)**
One request at a time	0.62 s	0.47 s
Requests that run at once	4 or more	2 (a setting; more needs more memory)
Phone photos, fields read exactly	**50 % (vision model not usable)**	**60 %**
Rulebook check with the model	15 of 15	15 of 15
Switching between models	Not possible	Automatic; the first photo after a long pause took about 70 s once (model loaded from disk); the photo test with swaps in between had a slowest photo of 8.8 s

> **Risks to keep in mind.** (1) Explanations are about 44 % slower under load; the margin over the 200 a minute target is small. (2) The 4-bit build is not the same file as the fp8 weights; the rule check passes, but the quality of the written explanations was not scored in either setup. (3) The vision model comes from a third-party conversion; the project's document 45 advises converting the official weights yourself and recording hashes before production. (4) The Ollama image is not pinned to a version yet. **When to go back to vLLM:** a card with 16 GB or more, or an explanation load above about 200 a minute.

3. How the tests work

The suite lives in `experiments/server/` and was run as `run_all.py --run server_20261101h` with the same targets as before (document 65 of the project). The 30-minute steady load, the restart test and all other experiments ran again in full.

Code	Question	Method
S1	How fast is the AI model?	Requests for explanations, 1 to 64 at the same time
S2	How fast is the whole claim pipeline?	Batches of 10 to 500 claims, then 30 minutes of steady load
S3	Does search by meaning find the right rule?	26 real questions and 5 off-topic strings
S4	Are documents read correctly?	40 test documents: digital PDF, image-only PDF, scan, photo
S5	Is anything lost when a service restarts?	Restart five services while claims arrive and count them
S6	What does the internet see?	Port scan, encryption test, headers, anonymous requests
S7	Is the audit trail intact?	Health panel, audit chain and ledger verification
S0	Are resources enough?	Memory, CPU and GPU sampled every five seconds during the run (507 samples, 42 minutes)

Notes on this run. (1) S1 was pointed at Ollama directly, because the script normally looks for the vLLM container. (2) The run started with about 8,500 test claims already stored, about five times as many as the first run started with (about 1,700), which matters for S2 and S0. (3) S3's speed test was blocked by the search rate limit (295 requests answered with HTTP 429), because the temporary high limit of the first run had been removed; the quality result is not affected. (4) The first attempt of S7 received HTTP 502 from the API; the check was repeated by itself a few minutes later (section 4). Only measured results are reported below.

4. Results, experiment by experiment

S1. The AI model on the GPU: pass, but slower than with vLLM

Each request asks the model for one explanation (about 250 tokens in, up to 160 out). With one request at a time the model answers in 0.47 seconds, faster than vLLM did (0.62 s). With more requests together, throughput rises to about 202 a minute at 2 requests and then stays flat at 198 to 210 a minute: Ollama runs 2 requests at once, and the rest wait in line. The best value, **209.9 a minute** at 16 requests, is 5 % above the target of 200. No request failed (560 requests).

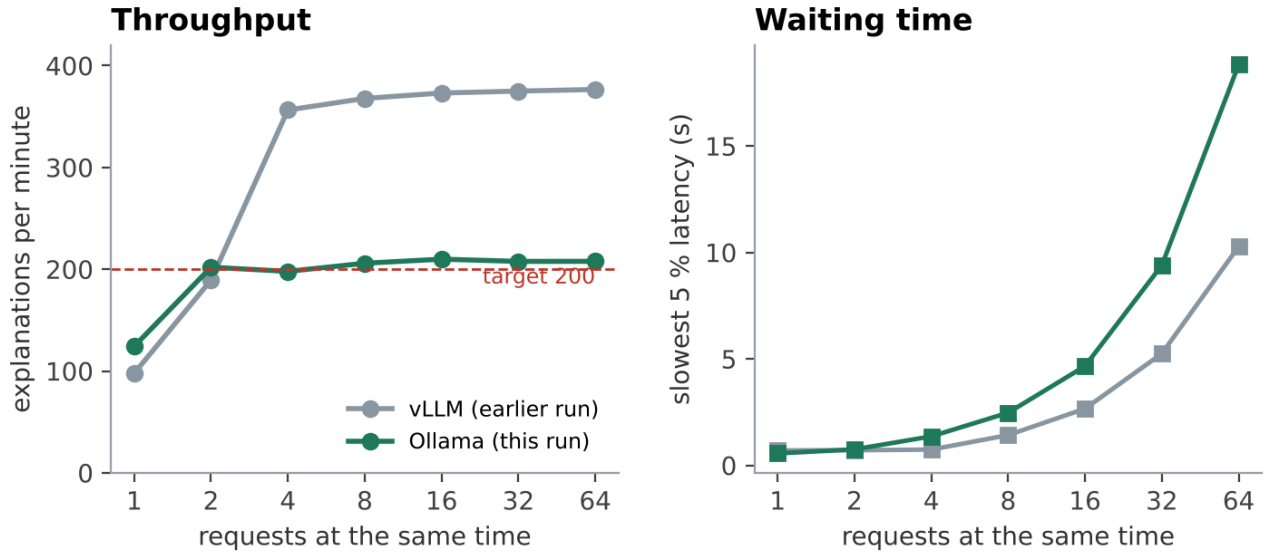


Figure 3. Explanations per minute (left) and waiting time of the slowest 5 % (right), with vLLM (grey, first run) and Ollama (green, this run).

At the same time	Requests	Errors	Per minute	Median (s)	Slowest 5 % (s)	Tokens/s
1	16	0	124.3	0.47	0.56	148.6
2	16	0	202.0	0.58	0.75	243.6
4	16	0	197.9	1.19	1.36	238.8
8	32	0	205.9	2.32	2.47	248.6
16	64	0	209.9	4.56	4.66	253.2
32	128	0	207.7	9.20	9.40	248.0
64	256	0	207.8	18.49	18.82	250.9

> **What it means.** The target is met, with little room. A reviewer waiting for one explanation gets it in about half a second; under heavy load the wait grows quickly (4.6 s at 16 requests). The GPU memory peak during this test was 4,379 MiB, so there is memory left to run more requests at once if needed (the setting is `OLLAMA_NUM_PARALLEL`), at the price of more memory.

S2. The claim pipeline: throughput passes, read pages are slow

Claims travel through proxy, API, queue, engine and store. The best rate was **4,140 claims a minute** with batches of 500, about 20 times the target and below the 11,721 of the first run. During the whole test the GPU was idle (0 % utilisation in the 30-minute steady load), so the explanation model is not

what limits this part. The most likely cause is the amount of data already stored: about 8,500 claims at the start (about 1,700 in the first run), with the API already using 3.6 of its 4 GiB. This was not proven.

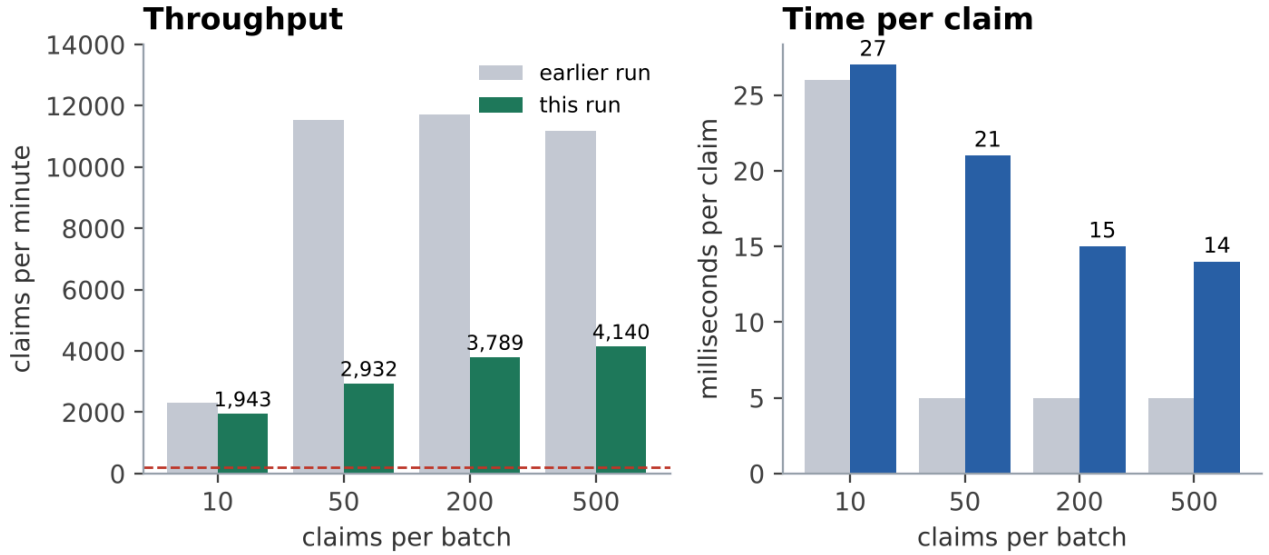


Figure 4. Claims per minute and milliseconds per claim by batch size: first run (grey) and this run (colour). Dashed line: target of 200 a minute.

Batch	Batches	Claims	Errors	Per minute	Batch done (s)	ms per claim
10	6	60	0	1,943	0.27	27
50	6	300	0	2,932	1.04	21
200	4	800	0	3,789	3.06	15
500	2	1,000	0	4,140	7.24	14

Reading the system, idle and right after a heavy load

This is the clearest change from the first run. In the first run the dashboard and the health panel answered in a few milliseconds when the system was idle. Now they take **about 2 seconds even when idle** (1.9 s and 2.0 s) and 2.6 to 2.7 s for the slowest 5 % right after the load, above the 2-second limit. The claims list stays fast (2 ms). The pages that count and summarise all stored claims are the slow ones, which points to the data volume and to the API running at its memory limit. A fresh database should be measured to confirm.

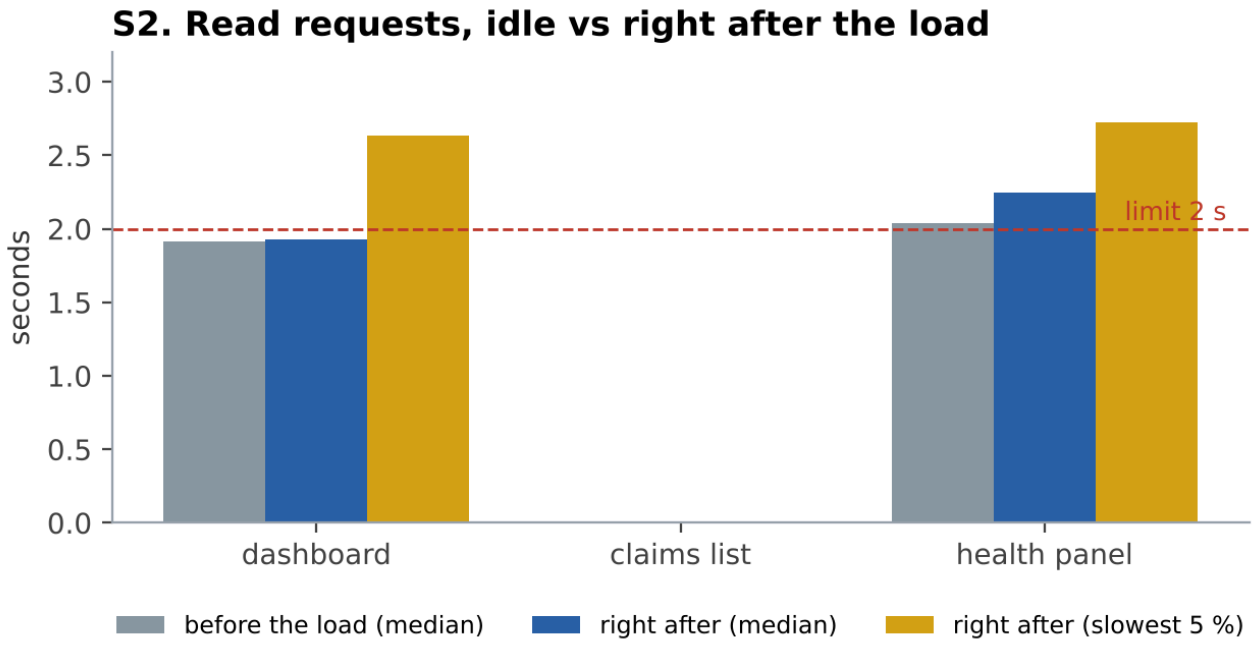


Figure 5. Answer time of three read requests, idle and right after the load. Limit: 2 seconds.

Steady load for 30 minutes: pass

3,000 claims were sent at 100 a minute for 30 minutes. **No batch failed.** The typical batch took 0.26 seconds throughout; the slowest 5 % reached 2.3 s once, at minute 12. The one failed batch of the first run did not repeat. This run recorded the error text of any failure, and there was none, so the earlier failure's cause remains unknown.

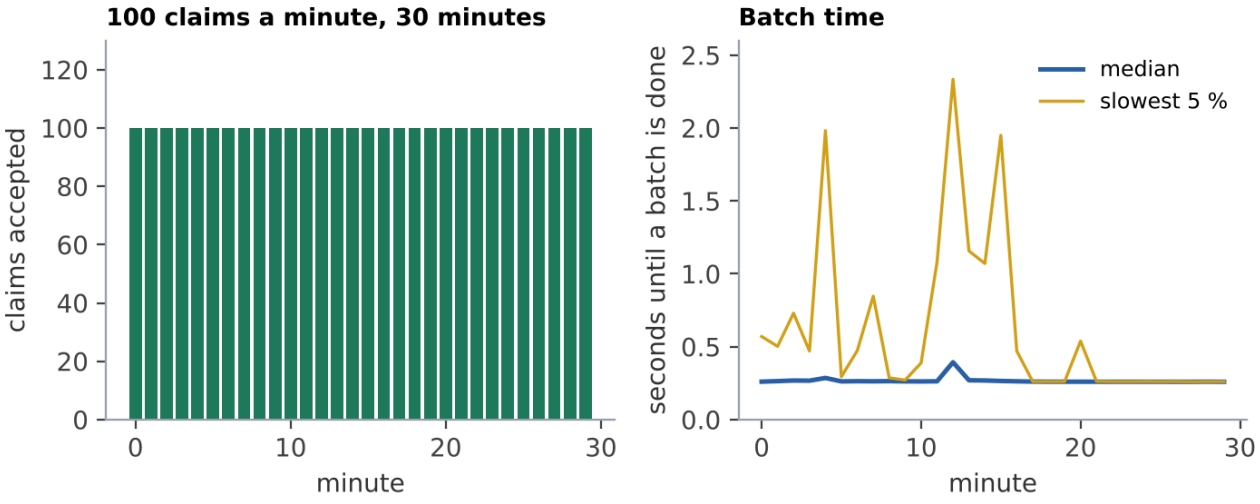


Figure 6. Claims accepted per minute (left) and time until a batch was done (right) during the 30-minute steady load.

S3. Search by meaning: quality passes, one off-topic string accepted (unchanged)

Staff type a question in their own words and the system finds the matching rule. **24 of 26 questions returned the right rule (92 %, target 90 %)**, the same as before. The same two questions missed: "claim line without a service date" (rule R001) and "how many days after the service can a claim be submitted" (rule R014 and two policy tables). **4 of 5 off-topic strings were refused**; one was answered, as before. The refusal level is a setting (`EMBEDDER_THRESHOLD=0.485`); raising it refuses more unrelated text but may also refuse genuine questions, and the pass line is only 2 questions away.

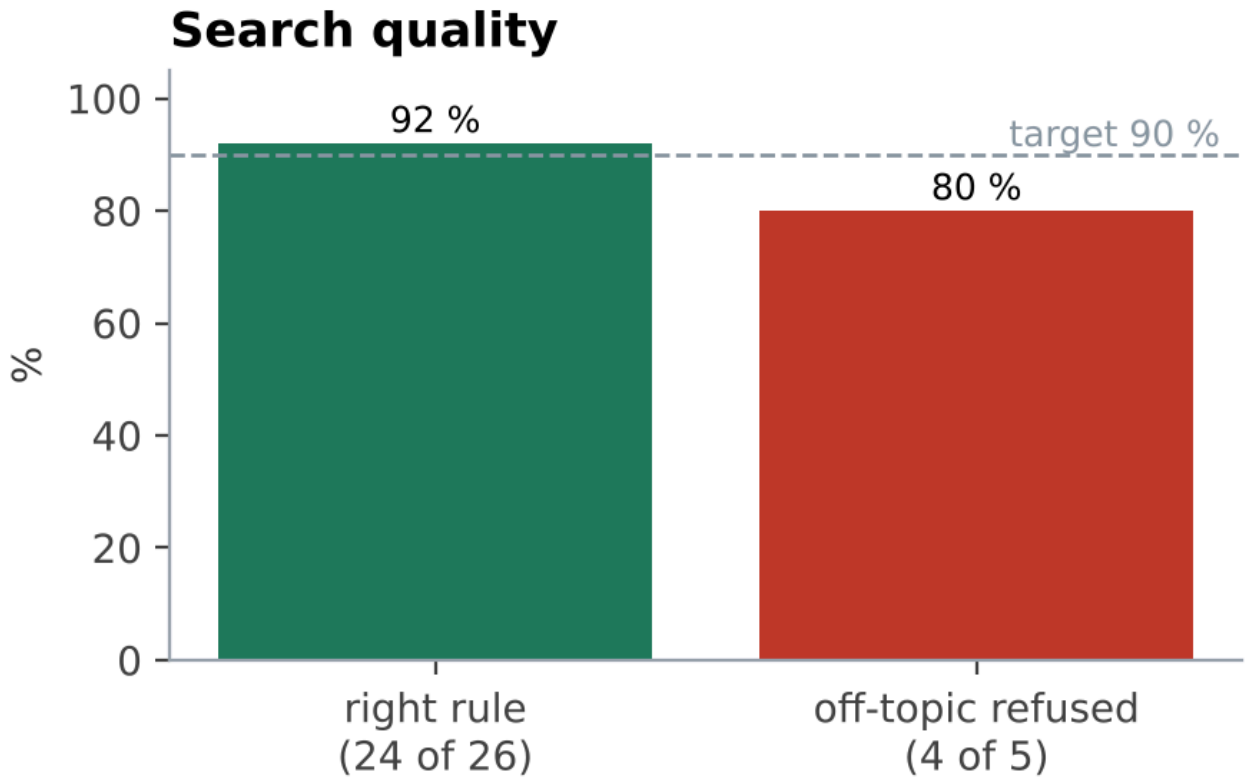


Figure 7. Share of questions answered with the right rule and share of off-topic strings refused.

Speed was not measured in this run. Of the speed test's requests, 295 were refused with HTTP 429 because the search rate limit was back at its normal value; only one level (1 at a time) produced answers. The first run's speed (a median of 2 to 4 ms, up to 1,819 answers a second at 8 at once) was measured with a temporary high limit and stands as the reference. The 26 quality questions took a median of 3.8 s each in this run (slowest 5 %: 5.2 s); the first report did not record this figure, so there is nothing to compare it to, and what the time consists of was not checked.

S4. Documents to claims: much better, photos still below target

40 documents were read, 0 failed to read. The reader now runs Tesseract and the vision model together and compares the two readings. The test with the vision model on the GPU was run three times with the same result (the last time with the explanation model loaded on the same card), so the figures are stable.

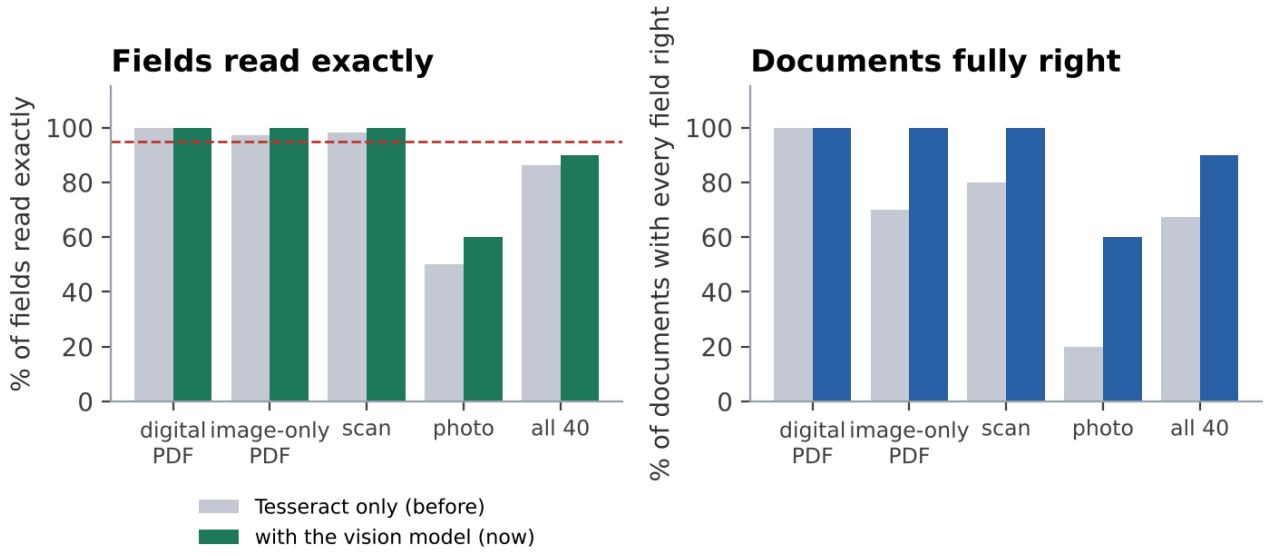


Figure 8. Left: share of fields read exactly, Tesseract only (grey) and with the vision model (green); red line: 95 % target. Right: share of documents with every field right.

Document type	Documents	Fields exact (before)	Fields exact (now)	Fully right (now)	Median time (s)
Digital PDF	10	100.0 %	100.0 %	100 %	0.0
Image-only PDF	10	97.3 %	100.0 %	100 %	3.1
Scan	10	98.2 %	100.0 %	100 %	3.2
Photo	10	50.0 %	**60.0 %**	**60 %**	2.3
All	40	86.4 %	**90.0 %**	**90 %** (was 67.5 %)	

What it means. PDFs and scans now meet the 95 % target in all three types. Phone photos improved but remain at 60 %, so the overall 90.0 % is still below the 95 % target. The project's own measurement of 88.1 % for photos came from its evaluation set on the build PC and is not directly comparable. For now, ask staff to submit PDFs or scans where possible, and keep a human check on every document read. A next step is to test a higher image resolution for the vision model (the server uses 130 dots per inch, lowered when memory was short; the project default is 200), since the card now has the room.

S5. Restarts under load: pass

Each service was restarted while a background client kept sending claims in batches of five. After each restart, the queue total was compared with the number of claims the system had acknowledged. A claim that was acknowledged and then missing would be the failure that matters.

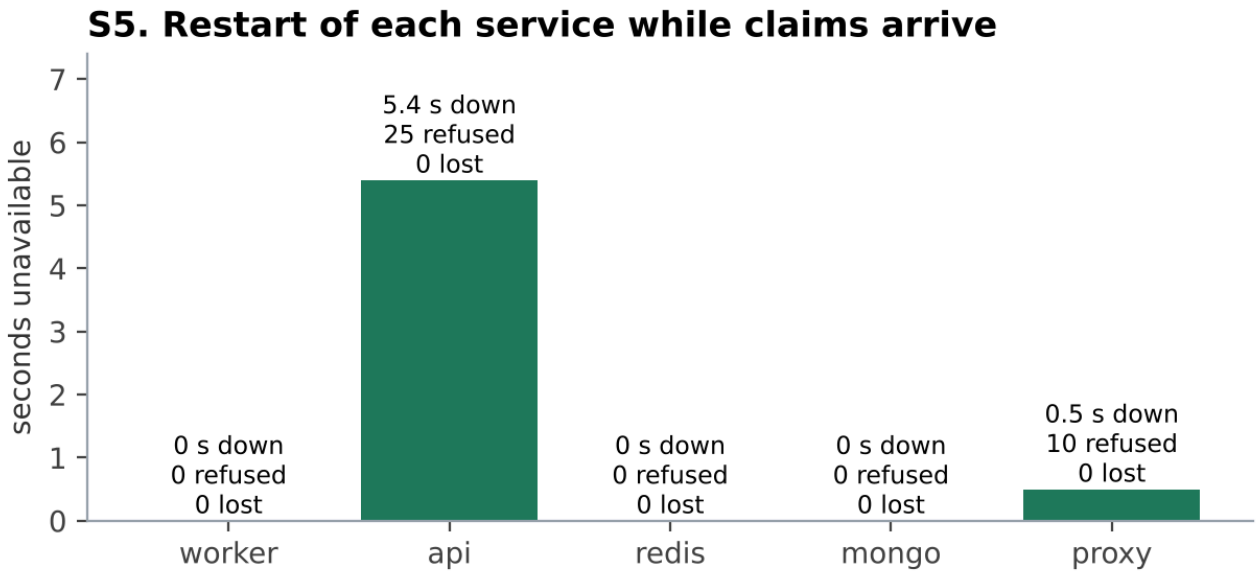


Figure 9. Seconds the service was unavailable, requests refused and claims lost, per restarted service.

Service	Restart (s)	Unavailable (s)	Back	Sent	Acknowledged	Refused	Lost
worker	2.7	0.0	yes	375	375	0	**0**
api	5.7	5.4	yes	280	255	25	**0**
redis	0.4	0.0	yes	335	335	0	**0**
mongo	0.5	0.0	yes	315	315	0	**0**
proxy	0.5	0.5	yes	185	175	10	**0**

What it means. The same result as before. Nothing acknowledged was lost and every service came back. Restarting the API makes the site unavailable for about 5 seconds, the proxy for half a second; requests sent in that window are refused, not silently dropped, so a client that retries loses nothing. Plan updates for a quiet moment.

S6. Exposure seen from the network: pass in practice

The scan looks at the server like an attacker would, from outside, and ran against the domain, so Cloudflare sits between the test and the server. The result is the same as in the first report: 7 of 8 checks pass.

Check	Result
HTTPS open	**Yes**
TLS 1.0 and 1.1	**Refused**
HTTP on port 80	**Redirects to HTTPS**
Security headers (HSTS, content-type protection, content security policy, framing protection, no server version)	**All present**
Anonymous request to the API and to the health panel	**Refused (401)**
`/healthz` shows only a minimal answer	**Yes**
Internal ports (database, Redis, model server, embedding, reader, Ollama, app, SSH) closed	**Reported as not all closed: Cloudflare answers on port 8080 (test artefact)**

> On the server itself, `ss -tlnp` shows only Caddy listening, on ports 80 and 443. Nothing listens on 8080, so the reading is Cloudflare's own port. Ollama's port 11434 is not published to the network either: it is reachable only from the other containers.

S7. Integrity and audit: pass, with one finding and one optional probe unknown

The first attempt, at the end of the document test, received **HTTP 502 from the API** on all four requests, so it tested nothing. The API was at its memory limit at that moment (4,103 of 4,096 MiB in the samples) and may have been restarting; this was not confirmed, but it is a real signal that the API was unavailable for a short time. The check was run again by itself a few minutes later, once the API had settled:

Check	Result
Health panel answers	**Yes (HTTP 200, 2.2 s)**
Audit chain verifies	**Yes: 19,197 events, key configured, anchor signed**
Ledger verifies	**Yes: 77 checkpoints over 19,199 log rows, signing key present**
Health probes	**Overall "degraded": every probe is "ok" except `timestamp` unknown (no external timestamp service, as in the first run) and `flow` degraded**

The `signals` probe, which was degraded in the first run because of failed test sign-ins, is "ok" now. The `flow` probe is new in this project version; what makes it degraded was not looked up, and it should be read in the health panel before real use.

S0. Resources during the run: warnings on two memory limits

Memory, CPU and GPU were sampled throughout the 42 minutes (507 samples). The server never had less than 3.4 GiB of its 16 GiB available; the busiest moment used 81 % of it. The GPU peaked at 5,909 MiB of 10,240 MiB (58 %), 100 % utilisation, 69 °C and 329 W, during the model test and the photo test.

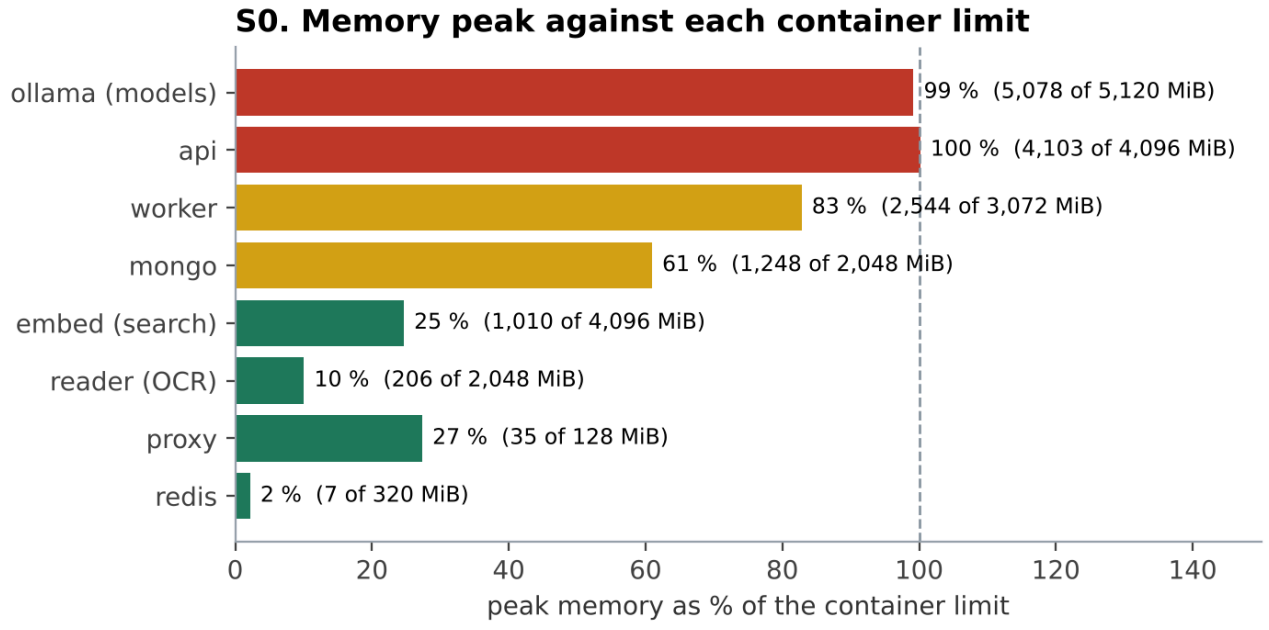


Figure 10. Peak memory of each container as a share of its limit.

Container	Peak (MiB)	Limit (MiB)	Share
ollama	5,078	5,120	**99 %**
api	4,103	4,096	**100 %**
worker	2,544	3,072	83 %
mongo	1,248	2,048	61 %
embed	1,010	4,096	25 %
reader	206	2,048	10 %
proxy	35	128	27 %
redis	7	320	2 %

The API is still the one to watch, and Ollama joins it. The API's limit was raised from 3 to 4 GiB after the first run, and it still touched the limit with about 15,000 test claims stored, because it keeps claims in memory. Its memory also grew by about 1.1 GiB an hour during the pipeline test while claims were added. A fresh database is the safe way to start and to see the real growth. Ollama's 99 % may be mostly file cache from reading the model files (not checked); if its container is ever killed (exit code 137), raise its limit.

5. Findings and recommendations

#	Finding	Cause	Recommended action	Priority
1	API answered HTTP 502 for a moment (first S7 attempt); health probe "flow" degraded	API at its memory limit, probably; flow cause not checked	Clean the database and repeat S7 and S0; read the "flow" probe in the health panel	**High**
2	API at 100 % of its 4 GiB; dashboard and health panel about 2 s when idle (S0, S2)	Claims kept in memory; about 15,000 stored test claims	Start with a clean database, then repeat S2 and S0 to see the real figures; watch `docker stats` weekly; raise the limit if the API passes 80 %	**High**
3	Photos read at 60 % (S4)	Vision model plus Tesseract still miss fields on blurred or tilted photos	Test a higher image resolution (`OCR_DPI` 200) and the project's prompt options; until then accept PDFs and scans and keep human review	**High**
4	Explanations 210 a minute, 44 % below vLLM (S1)	Ollama runs 2 requests at once and a 4-bit model	Enough for the 200 target, no more. Watch the queue; raise `OLLAMA_NUM_PARALLEL` if needed, or return to vLLM on a larger card	**Medium**
5	Ollama at 99 % of its memory limit (S0)	Probably model files cached in memory	Watch it; raise the limit (container and host have room: peak 81 % of 16 GiB) if the container is ever killed	**Medium**
6	One off-topic string answered (S3)	Refusal threshold lenient for that string	Test `EMBEDDER_THRESHOLD` slightly higher (for example 0.50) against the 26 questions and the 5 off-topic strings; the 24 of 26 pass line leaves no margin	**Medium**
7	Search speed not measured (S3, HTTP 429)	Normal rate limit active	Raise `ASK_PER_MINUTE` for the test only, run S3, then remove it again	**Low**
8	The failed batch of the first soak did not repeat (S2)	Unknown	Keep the recorded error text on; repeat the soak after clean-up	**Low**
9	API restart interrupts for about 5 s (S5)	Normal for one API instance	Apply updates in quiet hours; clients should retry refused requests	**Low**
10	Ollama image and vision model not pinned	Latest tag; third-party model conversion	Record the image digest and the model hashes; convert the official weights before production (document 45)	**Low**

6. Problems met and how each was solved

These are recorded so the next installation, update or rebuild after a failure goes faster. The first part is new in this edition (1 November); the second part is from the first installation.

6.1 Since the first report

Problem	Cause	Solution
Build froze the server (memory 98 %, swap full); Ctrl+C did nothing	Building the new image with the AI containers running on 12 GiB	Restart from Proxmox; raise memory to 16 GiB; stop the model and search containers before a build
Disk 100 % full (first time)	About 20 GB of build cache left by the rebuild	`docker builder prune -af` after every rebuild
Vision model failed on every photo (HTTP 500); photo score unchanged	Not enough GPU memory beside vLLM; the reader silently falls back to Tesseract and only logs a warning	Serve both models through Ollama (section 2.5). Check the reader's warnings when a score does not change
Disk 100 % full (second time); Ollama container 30 GB	Each out-of-memory crash of the vision model wrote a core dump inside the container	Remove the container (`docker compose rm -sf ollama`, without `-v`, so the model volume stays); add `ulimits: {core: 0}` to it
"The database cannot be reached"; API 503; sign-in failed	MongoDB crashed when the disk filled and stayed stuck; the API never reconnected	`docker compose restart mongo`, then `up -d` for the others. Data was intact
Vision model on the CPU: 2 to 4 minutes per photo; 39 of 40 documents failed	Test client waited 60 seconds	Run the model on the GPU (about 2 s per photo); the test client now waits 15 minutes
Model-check showed 0 of 15 right after a restart	The model was still loading	Wait until the container is healthy (vLLM) or for the first request to load the model (Ollama), then run it again
`docker compose` without the local settings in a new terminal	The command line is not stored between sessions	Define the compose command in each new shell, with `compose.local.yml` listed last
File copy into the reader container refused: read-only	The reader's filesystem is read-only on purpose	Test it from the host through the container's network address
S1 could not find the model server	The script looks for the vLLM container	Pass the model address (`--model-base`) and set `VLLM_MODEL=gemma3:4b`
Pasted Python edits did not match	PuTTY removes leading spaces when pasting	Search for text without leading spaces; check the count before replacing
`curl https://localhost/ui/` returns 000	Caddy answers by name or server address, not "localhost"	Use the server's IP address or the domain
Wrong file names when copying the update (`Claim_Guard-main (3).zip`)	Spaces and brackets in the name	Copy with the exact name in quotes

6.2 From the first installation

Problem	Cause	Solution
`curl` not found	Minimal system	Install curl and other basic tools with apt
GPU not usable in Docker (BPF error)	Proxmox container blocks cgroup control	Run the NVIDIA toolkit in no-cgroups mode
GPU libraries missing in containers	Versioned library files absent	Copy the `.so.1` files to versioned names; use CDI device access
Model download refused (403)	Gemma is a gated model; token empty	Accept the licence on Hugging Face; set the token silently; revoke a token that had been exposed
vLLM would not start (missing CUDA 13 libraries; compile error; no KV-cache memory)	Libraries not in the image; `/tmp` without exec; too little GPU memory reserved	Mount `/opt/nvrtc13` and set `LD_LIBRARY_PATH`; `tmpfs /tmp:size=4g,exec`; `VLLM_GPU_UTIL=0.85` (these concern the former vLLM setup)
Browser error SSL protocol	No default certificate name for access by IP	`default_sni` in Caddy
"That request came from another site"	The API compares the Origin header with the Host header; the proxy sent the internal name	`header_up Host claimguard.kernelsi.com` in each proxy block
Search service would not start (offline error)	Model name not resolvable offline	Point `EMBED_MODEL` to the local model folder
Test scripts blocked by Cloudflare (error 1010)	Python user agent refused	Run the scripts against the server's IP address
Site returned 502; API exit code 137	Memory limit 768 MiB too low for thousands of claims	Raise the API and worker limits (now 4 GiB and 3 GiB)

7. Checklist before real use

1. **Read the "flow" health probe** in the health panel and run S7 again after the database is cleaned (section 4, S7).
2. **Start clean.** Either a fresh database (removes about 15,000 test claims and all accounts; create the real admin with `cg admin create-admin`) or keep the data and delete the benchmark accounts CG-4002, CG-4003 and CG-1001 after a real administrator exists. A fresh database also lowers the API's memory use and probably the 2-second read pages. Repeat S2, S0 and S7 afterwards.
3. **Watch the disk.** The disk is 99 GB with 25 GB free. Run `docker builder prune -af` after any rebuild, check `df -h /` before long tests, and consider enlarging the virtual disk in Proxmox (the Proxmox host itself was 89 % full earlier: check it first).
4. **Protect the backups.** `backup-before-update/` contains the encryption key; store it, with the `.env` copy and `compose.local.yml`, off the Downloads folder (encrypted drive or password manager). Never commit them to GitHub.
5. **Delete the Hugging Face token** that is no longer needed, and keep SSH (port 22) closed to the internet. Consider limiting ports 80 and 443 to Cloudflare's address ranges.
6. **Delete temporary files** on the server: `~/bench.env`, `~/ph`, `~/rt.py`, `~/env.backup`, `~/compose.local.backup`, `~/results_h.tgz`, the update zip and temporary folder.
7. **Pin the Ollama image** (record its digest) and the model files.

8. **Schedule a backup** of the database and a restore test.

8. What this test does not cover

- All claims and documents were **synthetic**. Real hospital paper, handwriting and Arabic documents are untested.
- Results are for **one server** and one user at a time on the console. Many reviewers working together were not simulated, and photos and explanations were not sent at the same time in a mixed-load test (they were tested one after the other, with a model swap between).
- The time to swap between the explanation model and the vision model was not measured on its own.
- Accuracy of the AI explanations (are they correct and useful to a reviewer?) was not scored, with vLLM or with Ollama. S1 measures speed only.
- The search speed (S3) was not measured in this run, and the audit check (S7) passed only on a second attempt.
- Failure of the whole machine, loss of the disk and restoration from backup were not tested.
- Security testing here is a view from the network, not a penetration test.

9. Appendix

9.1 Files in the repository (branch khalil_test)

File	Content
`docs/66_Server_Experiment_Results.md`	Results as written by the script (rebuild it for run `server_20261101h`)
`docs/figures/server/`	Charts of the run
`docs/reports/ClaimGuard_Server_Report_v2.pdf`	This report (second edition)
`outputs/server/server_20261101h/`	Raw results (report.md, the experiment files and s06_exposure.json)
(not in Git) `deploy/rust/compose.local.yml`	The server's own settings; the project keeps it out of Git on purpose. Content in section 9.4

9.2 Commands used

Purpose	Command
The compose command of this server (define it in every new shell)	`DC="docker compose -f deploy/rust/compose.yml -f deploy/rust/compose.gpu.yml -f deploy/rust/compose.local.yml --env-file .env --profile documents --profile search"`
Download the explanation model into Ollama	`$DC --profile ollama-fetch run --rm ollama-fetch 'ollama serve >/tmp/o.log 2>&1 & sleep 8; ollama pull gemma3:4b; ollama list'`
Run all experiments (inside tmux, in the virtual environment)	`export VLLM_MODEL=gemma3:4b; python experiments/server/run_all.py --run server_20261101h --docs ~/docs_test --model-base http://<ollama address>:11434/v1 --soak-minutes 30 --yes-restart --compose "$DC"`
Run only the documents test	`python experiments/server/run_all.py --run server_20261101h --only s04 --docs ~/docs_test`
Integrity check on its own	`python experiments/server/s07_integrity.py --run server_20261101h`
Exposure test (from another machine)	`python experiments\server\s06_exposure.py --run server_20261101h --host claimguard.kernelsi.com`
Check the models answer	`$DC exec -T worker cg model-check | tail -2`
Free disk space after a build	`docker builder prune -af`
Listening ports on the server	`sudo ss -tlnp`

9.3 Glossary

Term	Meaning
Slowest 5 % (p95)	95 out of 100 requests were at least this fast
Median (p50)	The typical request: half were faster, half slower
OCR	Optical character recognition: reading text from an image
Vision model	An AI model that looks at an image and reads it; here the second reader for photos, next to Tesseract
vLLM	A program that runs an AI model on the GPU at high speed; it keeps most of the card reserved
Ollama	A program that runs AI models on the GPU and loads and unloads them as requests arrive, so several models can share one card
Core dump	A file a crashed program writes with a copy of its memory; it can be many gigabytes
TLS	The encryption behind HTTPS; versions 1.0 and 1.1 are obsolete
HSTS	A header that tells browsers to always use HTTPS
Soak test	A steady load over a long time to reveal slow leaks and rare failures
Exit code 137	The container was killed, usually for exceeding its memory limit
HTTP 429 / 502 / 503	Too many requests (rate limit) / the proxy could not reach the API / the service is unavailable

9.4 The server's own settings (`deploy/rust/compose.local.yml`, summarised)

Part	What it sets
api	Memory limit 4 GiB; explanation model address `http://ollama:11434`, model name `gemma3:4b`
worker	Explanation model address and name as for the api
reader	Vision model address `http://ollama:11434/v1`, model `richardyoung/unlimited-ocr`, page time limit 300 s, image resolution 130 dpi
ollama	Image `ollama/ollama:latest`; GPU access; models kept loaded for 30 minutes; one model loaded at a time; 2 requests at once; memory limit 5 GiB; no core dumps (`ulimits: core 0`); on the database and reader networks; model volume `ollama_data`
ollama-fetch	One-off container with internet access that downloads models into the volume (profile `ollama-fetch`)